

ADRs V2

Architecture Decision Records (ADR) — Index

This page provides a one-page index of all Architecture Decision Records for the Entity Resolution System (ERS). The ADRs are grouped by domain and stability to support Enterprise Architecture review, onboarding, and controlled evolution.

Foundational ADRs

These define client-facing meaning and non-negotiable invariants. Changes are exceptional and breaking.

Domain A — Identity Semantics and Client Guarantees

- **ADR-A1 — Canonical Identifier Derivation and Governance by Construction**
Defines canonical identifiers as opaque, deterministically derived identity anchors based on (sourceId, sourceRequestId, entityType).
- **ADR-A2 — Non-Empty Resolution Guarantee**
Guarantees that every valid resolution request yields a canonical identifier, including explicit singleton identities for no-match cases.
- **ADR-A3 — Identity Monotonicity**
Establishes that canonical identifiers are never revoked, even as assignments evolve.

Domain B — Authority and Governance Model

- **ADR-B1 — ERS-ERE Responsibility Split**
Establishes ERS as authoritative for identity, decisions, and governance, with ERE as advisory and reactive only.
- **ADR-B2 — Decision-Centric Governance Model**
Defines resolution decisions as the sole unit of governance, with exactly one active decision per request.
- **ADR-B3 — Curator Authority and Lock Semantics**
Makes curator confirmations and rejections immutable locks that override automation.
- **ADR-B4 — Normative Confidence Semantics**
Defines confidence as a discrete governance signal (+1, 0, -1), distinct from engine similarity scores.

Structural ADRs

These shape architecture and operational correctness. Changes require careful migration planning.

Domain C — Distributed Interaction and Failure Semantics

- **ADR-C1 — Asynchronous REST and Eventual Consistency Client Interaction**
Defines immediate REST responses with asynchronous processing and read-only lookup for convergence.
- **ADR-C2 — ERS–ERE Messaging and Delivery Guarantees**
Establishes asynchronous, at-least-once messaging with idempotency, no ordering assumptions, and request-level correlation.
- **ADR-C3 — Late Response, Duplicate Response, and Error Absorption Rules**
Normalises lateness, duplication, and engine errors while preserving governance guarantees.

Domain D — State Authority and Persistence Model

- **ADR-D1 — State Separation and Authoritative Stores**
Separates immutable intake, governance state, and canonical serving state with clear authority boundaries.

Domain E — Corrective Operations and Consistency Preservation

- **ADR-E1 — Rebuild Semantics and Governance Preservation**
Defines rebuild as an explicit, scoped, admin-triggered corrective operation that preserves locks and identity invariants.
- **ADR-E2 — Re-Resolution with Rejections Protocol**
Separates read-only lookup from explicit re-resolution with rejection constraints.

Policy ADRs

These are intentionally revisitable and isolated to avoid contaminating foundational invariants.

Domain F — Representation and Payload Boundaries

- **ADR-F1 — Representation Symmetry at the ERS–ERE Boundary**
Requires ERS and ERE to operate on the same domain-driven, engine-agnostic representations.
- **ADR-F2 — No Canonical Representation Materialisation by ERS**
Restricts ERS to identity anchoring and forbids authoritative canonical payloads.
- **ADR-F3 — UI Preview Extraction is Non-Authoritative**
Isolates UI preview data as best-effort, presentation-only information.
- **ADR-F4 — Candidate Handling and Top-N Pruning Policy**
Centralises candidate pruning and threshold usage within ERS, forbidding boundary-level filtering.

Domain G — Versioning and Lineage Policy

- **ADR-G1 — No Exposed Version History or Lineage Graph for Canonical Entities**

Restricts clients to the latest canonical state and forbids exposed lineage or version history.

Domain H — Observability and Telemetry

- **ADR-H1 — Observability and Telemetry Semantics**

Defines best-effort, metadata-only observability that does not affect system behaviour or contracts.

Domain I — Authentication and authorisation

ADR-I1 — Authentication via External SSO; ERS Retains Authorisation Only

-ADR template

You are writing an Architecture Decision Record (ADR).

Your goal is to maximise clarity, normativity, and long-term usefulness, while minimising verbosity.

The ADR is a decision artifact, not a design narrative.

Follow the template and writing rules below strictly.

Architecture Decision Record (ADR)

ADR ID and Title

State the decision in one clear, precise sentence.

Context and Problem Statement

Explain why a decision is required now.

Focus on constraints, tensions, and invariants that make the decision necessary.

Do not explain history or tell a story.

Decision

State what has been decided in prescriptive terms.

Use clear rules and invariants.

Explicitly state what is allowed and what is not allowed after this decision.

Scope and Applicability

Define where this decision applies and where it does not.

Mention affected systems, components, interfaces, or interactions.

Explicitly state exclusions if relevant.

Considered Options

List only realistic alternatives that were seriously considered.

Keep this short; do not justify each option here.

Rationale

Explain why the chosen option is correct given the context.

Focus on trade-offs and optimisation goals, not on persuasion.

Avoid repeating the context or the decision.

Consequences

Describe the effects of this decision.

Include both benefits and costs.

State new obligations, constraints, or follow-up work introduced by the decision.

Writing Rules (Mandatory)

- Do not include status, timestamps, authors, or change rules.
- Do not use horizontal lines or visual separators between sections.
- Use Markdown headings only for section titles.
- Use bullet points only when they add clarity.
 - Short enumerations may be inline, separated by commas.
 - Longer or normative enumerations must be bullet points.
- Bullet points must express short phrases or full sentences, not single words.
- Prefer normative language ("must", "shall", "is", "is not") over explanatory language.
- Avoid repetition across sections; each section must add new information.
- Avoid storytelling, historical narration, and emotional justification.
- Keep the ADR readable by skimming; it should be understandable without deep domain immersion.

The resulting ADR should read as a stable architectural rule set that implementers and reviewers can rely on directly.

ADR-A1 — Canonical Identifier Derivation and

ADR-A1 — Canonical Identifier Derivation and Governance by Construction

Context and Problem Statement

- The system must expose stable canonical identifiers that can be relied upon by clients, integrators, and downstream systems across asynchronous processing and long-lived data flows.
- Identity stability must not depend on runtime coordination, engine-specific behaviour, or implicit authority assumptions.
- ERS and ERE must interoperate using the same request-level identifiers without introducing hidden coupling or lifecycle complexity.

Decision

- Canonical identifiers are opaque identifiers with no URI, HTTP, or dereferenceability semantics.
- Resolution requests and ERS–ERE exchanges are identified by the tuple (sourceId, sourceRequestId, entityType), which is treated as the stable mention-level identifier set across contracts.
- A canonical identifier is deterministically derived from the tuple (sourceId, sourceRequestId, entityType) using a specification-defined derivation rule. For instance, a canonical identifier might be built by some string composition like “can:\$entityType:\$sourceId:\$sourceRequestId”
- The derivation rule itself is the governance mechanism and shall be implemented consistently by ERS and by all ERE implementations.
- ERS is authoritative for exposing canonical identifiers to clients but shall not mint, alter, or override identifiers outside the deterministic derivation rule.

Scope and Applicability

- This decision applies to all external APIs, message contracts, persistence models, and documentation that reference canonical identity or mention-level identification.
- It governs both ERS–client interaction and ERS–ERE interaction.
- Representation formats, encoding choices, and any future URI serialisation are explicitly out of scope.

Considered Options

- Centralised runtime minting of canonical identifiers by ERS.
- Engine-owned or engine-specific identifier generation.
- Deterministic derivation based on stable request-level identifiers.

Rationale

- Deterministic derivation guarantees identity stability by construction rather than by authority or coordination.
- Using the same mention-level identifiers across all interactions eliminates translation layers and correlation ambiguity.
- Opaque identifiers avoid unintended semantic commitments and preserve architectural flexibility.

Consequences

- Identifier stability is guaranteed as long as input identifiers and the derivation rule remain unchanged.
- ERS and ERE implementations must strictly conform to the same derivation specification and identifier usage.
- Any change to the derivation rule or mention-level identifier semantics constitutes a foundational, breaking change.

ADR-A2 — Non-Empty Resolution Guarantee

ADR-A2 — Non-Empty Resolution Guarantee

Context and Problem Statement

- Client systems require a usable identifier for every valid resolution request in order to maintain processing continuity and avoid conditional or error-driven logic.
- Asynchronous resolution, partial information, or low-confidence matching must not result in missing or null identity outcomes.
- Identity availability must be guaranteed independently of resolution quality, engine behaviour, or curation state.

Decision

- Every valid resolution request shall yield a canonical identifier.
- The absence of a match is represented explicitly as a singleton canonical entity, not as null, empty, or missing identity.
- A singleton canonical entity represents the entity as unresolved-with-others and remains a valid, stable identity anchor.

Scope and Applicability

- This decision applies to all ERS-facing APIs, client contracts, and downstream integrations that consume resolution results.
- It governs both initial resolution submissions and any subsequent re-resolution operations.
- Error handling for invalid requests and transport-level failures is out of scope.

Considered Options

- Returning null or no identifier when no match is found.
- Deferring identifier issuance until a match is discovered.
- Representing no-match cases as explicit singleton canonical entities.

Rationale

- A non-empty identity guarantee simplifies client logic and integration contracts.
- Explicit singleton identities preserve semantic clarity without introducing special cases.
- Decoupling identity availability from match quality supports asynchronous processing and governance evolution.

Consequences

- Clients can rely on the presence of a canonical identifier for all valid requests.
- The system may accumulate singleton canonical entities that are never merged with others.
- Resolution quality improvements affect assignments, not the existence of identity.

ADR-A3 — Identity Monotonicity

ADR-A3 — Identity Monotonicity

Context and Problem Statement

- Client systems and downstream datasets may retain references to canonical identifiers for long periods of time.
- Resolution outcomes may evolve as new information arrives or curation actions are applied.
- Identity evolution must not break existing references or require clients to handle identifier invalidation.

Decision

- Canonical identifiers are never revoked, invalidated, or deleted once they exist.
- Assignments between requests and canonical identifiers may evolve over time, and the canonical identifier observed by a client for a given request may change.
- Identity stability is defined at the level of canonical identifier existence, not at the level of representations, assignments, or resolution state.

Scope and Applicability

- This decision applies to all canonical identifiers exposed by ERS and referenced by clients or downstream systems.
- It governs resolution, re-resolution, curation, and rebuild operations.
- Representation changes, storage optimisation, and internal housekeeping are out of scope as long as identifier existence is preserved.

Considered Options

- Allowing revocation or invalidation of canonical identifiers.
- Introducing versioned or lineage-based canonical identifiers.
- Preserving canonical identifiers permanently while allowing assignment changes.

Rationale

- Monotonic identity guarantees protect long-lived references and downstream data stability.
- Separating identifier existence from assignment evolution avoids cascading changes across systems.
- Avoiding revocation and lineage semantics keeps client contracts simple and predictable.

Consequences

- The system may retain canonical identifiers that are no longer actively referenced.
- Clients must treat canonical identifiers as stable references, not as indicators of current resolution state.
- Resolution improvements affect which identifier is observed, not which identifiers exist.

ADR-B1 — ERS–ERE Responsibility Split

ADR-B1 — ERS–ERE Responsibility Split

Context and Problem Statement

- The system integrates one or more Entity Resolution Engines (EREs) whose implementations may vary and be replaced over time.
- Governance decisions, identity guarantees, and lifecycle management must remain stable regardless of engine behaviour or availability.
- Clear authority boundaries are required to avoid implicit coupling, duplicated responsibility, or conflicting sources of truth.

Decision

- ERS is authoritative for exposing canonical identifiers, managing decision lifecycles, and enforcing governance guarantees.
- ERE performs resolution computation only when invoked by ERS and produces advisory results.
- ERE has no independent authority over identifier exposure, decision state, governance semantics, or client-facing behaviour.
- ERE has no autonomous lifecycle and does not initiate resolution, re-resolution, or corrective operations on its own.

Scope and Applicability

- This decision applies to all ERS–ERE interactions, contracts, and integration points.
- It governs runtime behaviour, operational responsibilities, and ownership of system guarantees.
- Internal implementation choices within ERS or ERE are out of scope as long as authority boundaries are respected.

Considered Options

- Granting ERE partial authority over identity or decision outcomes.
- Allowing ERE to initiate resolution or corrective operations independently.
- Treating ERE outputs as advisory inputs governed by ERS.

Rationale

- Centralising authority in ERS preserves stable client-facing guarantees.
- Treating ERE as advisory enables engine replaceability and controlled evolution.
- Clear responsibility boundaries reduce coordination complexity and governance ambiguity.

Consequences

- ERS must validate, interpret, and govern all resolution outcomes produced by ERE.
- ERE implementations can evolve or be replaced without changing external contracts.
- All client-visible behaviour and guarantees remain consistent even under engine failure or change.

ADR-B2 — Decision-Centric Governance Model

ADR-B2 — Decision-Centric Governance Model

Context and Problem Statement

- The system must support automated resolution, human curation, and iterative improvement without conflating computation with authority.
- Governance actions must be traceable, reviewable, and stable even as resolution algorithms and candidate sets evolve.
- Applying governance directly to entities or clusters risks ambiguity, hidden state changes, and loss of accountability.

Decision

- Governance applies to resolution decisions, not to canonical entities or clusters as such.
- For each resolution request identified by (sourceId, sourceRequestId, entityType), there is exactly one active decision at any point in time.
- Decisions progress through an explicit lifecycle, such as proposed, accepted, rejected, or confirmed.
- Curator actions are authoritative state changes within the decision lifecycle and override automated outcomes.

Scope and Applicability

- This decision applies to all resolution, re-resolution, curation, and corrective workflows managed by ERS.
- It governs how automated outputs, human actions, and governance state interact.
- Internal engine scoring, candidate generation, and representation details are out of scope.

Considered Options

- Applying governance directly to canonical entities or clusters.
- Allowing multiple concurrent active decisions for the same request.
- Treating automated resolution outputs as authoritative outcomes.

Rationale

- Decision-centric governance provides a clear unit of authority and accountability.

- A single active decision per request avoids conflicting or ambiguous governance states.
- Explicit lifecycles enable controlled automation while preserving human oversight.

Consequences

- Canonical entities may exist independently of the current governance state.
- All governance logic, auditing, and curation operate at the decision level.
- Changes in resolution logic affect proposed decisions, not governance invariants.

ADR-B3 — Curator Authority and Lock Semantics

ADR-B3 — Curator Authority and Lock Semantics

Context and Problem Statement

- The system supports both automated resolution and human curation, with different expectations of authority and accountability.
- Human decisions must remain stable over time, even as automated processes, engines, or data inputs evolve.
- Without explicit lock semantics, automated re-resolution or rebuild operations risk silently overriding curator intent.

Decision

- Curator confirmations (+1) and rejections (−1) are immutable locks applied at the resolution decision level.
- Locked decisions shall not be overridden, weakened, or reverted by automated resolution outputs.
- Locks persist across re-resolution and rebuild operations and remain effective until explicitly changed by a curator.
- All automated processes must treat curator locks as hard constraints.

Scope and Applicability

- This decision applies to all curation workflows, automated resolution processes, and corrective operations.
- It governs how human actions interact with automation within ERS.
- User interface behaviour and access control mechanisms are out of scope.

Considered Options

- Allowing automated processes to override curator decisions under certain conditions.
- Treating curator actions as advisory signals rather than hard locks.
- Enforcing immutable locks at the decision level.

Rationale

- Immutable locks preserve human authority and accountability.
- Explicit lock semantics prevent accidental erosion of curated outcomes.
- Treating curator actions as hard constraints simplifies governance reasoning and auditability.

Consequences

- Automated resolution quality improvements cannot affect locked decisions without explicit curator intervention.
- Rebuild and re-resolution processes must be designed to respect existing locks.
- Curator actions have long-lived impact and must be applied deliberately.

ADR-B4 — Normative Confidence Semantics

ADR-B4 — Normative Confidence Semantics

Context and Problem Statement

- Resolution processes often expose numerical scores whose meaning varies across engines and algorithms.
- Clients and curators require stable, interpretable signals that reflect governance state rather than computational heuristics.
- Conflating similarity metrics with governance confidence leads to misinterpretation, overfitting, and brittle contracts.

Decision

- Confidence is a governance state signal, not a probability or similarity measure.
- The normative confidence values are +1 (confirmed), 0 (unconfirmed or neutral), and -1 (rejected).
- Confidence values reflect governance and curation state and are controlled by ERS.
- Engine similarity scores, heuristics, or internal metrics are non-normative and are not part of the conceptual model or external contract.

Scope and Applicability

- This decision applies to all client-facing APIs, curation interfaces, and governance logic that expose or interpret confidence.
- It governs how resolution outcomes are classified and acted upon.
- Internal engine scoring models and experimentation are out of scope.

Considered Options

- Exposing probabilistic confidence values derived from engine scores.
- Treating engine similarity scores as governance confidence.
- Defining a small, discrete set of normative confidence states.

Rationale

- Discrete governance states provide stable semantics independent of engine implementation.
- Separating governance confidence from similarity metrics preserves engine replaceability.
- Explicit confidence semantics reduce ambiguity in curation and downstream usage.

Consequences

- Confidence values cannot be interpreted as probabilities or likelihood estimates.
- UI filtering, workflow routing, and automation must rely on normative confidence states.
- Engine-specific scores may be used internally but must not leak into contracts or governance decisions.

ADR-C1 — Asynchronous REST

ADR-C1 — Asynchronous REST and Eventual Consistency Client Interaction

Context and Problem Statement

- Resolution involves asynchronous processing that may take variable time depending on engine behaviour, data quality, and system load.
- Client systems require predictable API behaviour that does not depend on engine responsiveness.
- Mixing synchronous waiting with asynchronous processing risks timeouts, hidden coupling, and inconsistent client expectations.

Decision

- ERS REST submission returns immediately for every valid request with a draft canonical identifier.
- Resolution processing by ERE is asynchronous and decoupled from the REST request lifecycle.
- Clients observe resolution convergence exclusively through explicit, read-only lookup operations.
- The ERS API does not provide blocking resolution, waiting semantics, or configurable time budgets.

Scope and Applicability

- This decision applies to all ERS client-facing REST APIs involved in resolution submission and lookup.
- It governs client interaction patterns and API guarantees.
- Internal orchestration mechanisms and messaging technologies are out of scope.

Considered Options

- Blocking REST calls until engine resolution completes.
- Hybrid APIs with optional waiting or timeout parameters.
- Immediate response with asynchronous processing and explicit lookup.

Rationale

- Immediate responses provide stable latency and availability guarantees to clients.
- Explicit asynchrony avoids implicit coupling between client behaviour and engine performance.

- Read-only lookup operations make eventual consistency visible and controllable.

Consequences

- Clients must adopt polling or follow-up lookup patterns to observe resolution changes.
- API semantics are simple and uniform across implementations.
- Resolution latency improvements do not affect client-facing contracts.

ADR-C2 — ERS–ERE Messaging and Delivery Guarantees

ADR-C2 — ERS–ERE Messaging and Delivery Guarantees

Context and Problem Statement

- ERS and ERE interact in a distributed environment where message delivery may be delayed, duplicated, or reordered.
- Resolution correctness must not depend on perfect delivery, strict ordering, or synchronous coupling.
- Messaging guarantees must be explicit so that implementations handle failure modes consistently.

Decision

- ERS–ERE interaction uses asynchronous messaging via a broker, with the concrete technology abstracted at this level.
- Message delivery is at-least-once; duplicate messages are expected and must be tolerated.
- All message consumers must be idempotent.
- ERS and ERE exchange and correlate messages using the same mention-level identifiers as provided in the original client request, namely the tuple (sourceId, sourceRequestId, entityType).
- Message ordering is not assumed and must not be relied upon.

Scope and Applicability

- This decision applies to all ERS–ERE message exchanges related to resolution and re-resolution.
- It governs delivery expectations, consumer behaviour, and correlation semantics.
- Broker configuration details and operational tuning are out of scope.

Considered Options

- Synchronous communication between ERS and ERE.
- Exactly-once delivery guarantees.
- Relying on ordered message streams.

Rationale

- At-least-once delivery provides robustness without excessive infrastructure complexity.

- Idempotent consumers allow safe retries and failure recovery.
- Explicit correlation and lack of ordering assumptions simplify distributed reasoning.

Consequences

- Implementations must handle duplicate and out-of-order messages correctly.
- Resolution logic must be resilient to partial failures and retries.
- Messaging infrastructure can be changed without affecting business semantics.

ADR-C3 — Late Response, Duplicate Response

ADR-C3 — Late Response, Duplicate Response, and Error Absorption Rules

Context and Problem Statement

- Distributed resolution processing produces responses that may arrive late, out of order, or be duplicated.
- Engine failures or partial results must not compromise client-facing guarantees or governance correctness.
- Failure handling rules must be explicit to avoid implicit coupling to timing or delivery assumptions.

Decision

- Late and out-of-order responses from ERE are expected and treated as normal system behaviour.
- Duplicate responses are tolerated through mandatory idempotent processing.
- Engine errors are absorbed by ERS and do not surface as violations of client-facing identity or availability guarantees.
- The latest applicable result is applied, subject to curator locks and the active decision lifecycle constraints.

Scope and Applicability

- This decision applies to all ERS handling of ERE responses during resolution, re-resolution, and rebuild operations.
- It governs error handling, response ordering, and state updates under failure conditions.
- Transport-level retry policies and engine-internal error handling are out of scope.

Considered Options

- Rejecting late or out-of-order responses as invalid.
- Propagating engine errors directly to clients.
- Absorbing failures and applying results opportunistically under governance constraints.

Rationale

- Treating lateness and duplication as normal simplifies distributed reasoning.

- Absorbing engine errors preserves stable client guarantees and decouples availability from engine health.
- Applying results under explicit governance constraints ensures correctness without over-synchronisation.

Consequences

- ERS must maintain sufficient state to evaluate applicability of incoming responses.
- Engine behaviour and timing variations do not affect external contracts.
- Governance invariants remain enforceable even under degraded or partial engine operation.

ADR-D1 — State Separation and Authoritative Stores

ADR-D1 — State Separation and Authoritative Stores

Context and Problem Statement

- The system maintains multiple kinds of state with different lifecycles, authority models, and access patterns.
- Mixing intake data, governance context, and served canonical state risks ambiguity, accidental overwrites, and unclear responsibility boundaries.
- Storage design must support auditability, correctness, and operational simplicity without over-constraining implementation choices.

Decision

- The System of Record is the authoritative store for intake data and representations and follows immutable, append-only semantics.
- The Decision Store is the authoritative store for governance context, candidate sets, decision lifecycle state, and decision rationale.
- The Canonical Entity Registry is the authoritative store for the latest accepted canonical state required for lookup and serving.
- Event sourcing is not required.
- Internal persistence mechanisms are implementation-specific, but authority boundaries between stores shall be preserved.

Scope and Applicability

- This decision applies to all persistence layers and stateful components within ERS.
- It governs data ownership, update rights, and read responsibilities across the system.
- Physical storage technologies, schemas, and performance optimisations are out of scope.

Considered Options

- Using a single unified store for all system state.
- Applying event sourcing as a universal persistence model.
- Separating state by authority and lifecycle.

Rationale

- Separating state by authority clarifies responsibilities and prevents accidental coupling.

- Append-only intake preserves auditability without requiring full event sourcing.
- Isolating canonical serving state simplifies lookup semantics and operational behaviour.

Consequences

- Implementations must respect clear ownership boundaries when reading or mutating state.
- Cross-store coordination must be explicit rather than implicit.
- Storage layers can evolve independently as long as authority semantics are maintained.

ADR-E1 — Rebuild Semantics

ADR-E1 — Rebuild Semantics and Governance Preservation

Context and Problem Statement

- Over time, resolution quality, engine logic, or data completeness may require corrective processing across existing decisions.
- Corrective operations must not undermine established governance guarantees or client-facing identity stability.
- Operators need a controlled mechanism to trigger corrective actions without introducing hidden side effects.

Decision

- Rebuild is an explicit, administrator-triggered corrective operation.
- Rebuild execution is asynchronous, best-effort, and resumable.
- Rebuild scope is explicit and bounded, at least by entity type and optionally by a time window or request subset.
- Curator locks and identity invariants are preserved during rebuild operations.
- Rebuild may request re-resolution from ERE with explicit rejection constraints derived from existing governance state.

Scope and Applicability

- This decision applies to operational and administrative workflows that perform corrective processing.
- It governs how rebuild interacts with governance state, identity guarantees, and engine invocation.
- User-facing curation workflows and client APIs are out of scope.

Considered Options

- Implicit or automatic rebuild triggered by system changes.
- Rebuild operations that reset governance or identity state.
- Explicit, scoped rebuild that preserves governance invariants.

Rationale

- Explicit triggering prevents accidental large-scale state changes.
- Asynchronous and resumable execution supports operational safety.
- Preserving locks and identity invariants maintains trust in curated outcomes.

Consequences

- Operators must define rebuild scope deliberately before execution.
- Rebuild operations may take time and produce partial progress.
- Corrective improvements do not invalidate prior governance decisions.

ADR-E2 — Re-Resolution with Rejections Protocol

ADR-E2 — Re-Resolution with Rejections Protocol

Context and Problem Statement

- The system needs both observation operations and corrective operations, and they must not be conflated.
- If lookup triggers re-processing implicitly, clients and operators cannot reason about side effects, load, or governance outcomes.
- Corrective re-resolution must respect prior governance outcomes, especially curator rejections.

Decision

- Re-resolution is an intentional, side-effectful operation and is distinct from lookup.
- Lookup operations are strictly read-only and never trigger re-resolution or any other state changes.
- Re-resolution requests may include explicitly rejected canonical identifiers as hard constraints.
- ERS is responsible for deriving and providing rejection constraints to ERE when applicable.

Scope and Applicability

- This decision applies to ERS operational APIs and ERS–ERE interactions that initiate corrective re-processing.
- It governs the semantics of lookup versus corrective operations.
- Transport mechanisms and UI labelling are out of scope.

Considered Options

- Combining lookup and re-resolution in a single operation.
- Allowing lookup to opportunistically trigger re-resolution.
- Strict separation between read-only lookup and explicit re-resolution.

Rationale

- Separating observation from correction prevents hidden side effects and unpredictable load.
- Explicit rejection constraints preserve governance intent while enabling corrective processing.

- Clear semantics simplify client expectations and operational procedures.

Consequences

- Clients and operators must invoke re-resolution explicitly when corrective processing is desired.
- The contract must support conveying rejected identifiers when re-resolution is requested.
- Read-only lookup remains safe to call frequently without triggering processing.

ADR-E3 — Best-Effort Projection

ADR-E3 — Best-Effort Projection of Curated Resolution Constraints to Entity Resolution Engines

Context and Problem Statement

ERS maintains authoritative Resolution Decisions, including curator-confirmed positive and negative constraints. Automated resolution engines may repeatedly propose mappings that have already been manually rejected, increasing curator workload and integration noise. A decision is required on whether, and how, curator-confirmed constraints are made available to Entity Resolution Engines without weakening ERS authority, breaking deterministic identity rules, or introducing hard coupling between ERS and ERE.

Decision

ERS may project curator-confirmed resolution constraints to Entity Resolution Engines as non-authoritative, best-effort inputs.

The following rules apply:

- ERS remains the sole authority for Resolution Decisions and canonical mappings.
- Projected constraints are advisory inputs; EREs may consume or ignore them.
- Correctness, identity guarantees, and governance outcomes shall not depend on ERE having received or applied projected constraints.
- Absence, delay, or loss of projected constraints shall not block resolution, rebuild, or lookup operations.
- Curated constraints shall always be enforced during decision integration within ERS, regardless of ERE behaviour.

Scope and Applicability

This decision applies to the ERS–ERE interaction boundary, including:

- optional constraint feeds or messages published by ERS,
- ERE consumption of curator-confirmed accept and reject constraints.

This decision does not apply to:

- client-facing APIs,
- canonical identifier derivation rules,
- decision persistence or governance logic inside ERS.

Considered Options

- Do not project curated constraints to ERE.
- Project curated constraints to ERE as best-effort, non-authoritative inputs.
- Require synchronous or mandatory synchronisation of curated constraints before resolution or rebuild.

Rationale

Best-effort projection reduces repeated invalid proposals and curator fatigue while preserving strict governance boundaries. Making constraint projection optional avoids coupling ERS correctness to ERE state and preserves engine replaceability. Mandatory synchronisation was rejected due to increased operational coupling and risk of cascading failure.

Consequences

Positive:

- Reduced repetition of curator-rejected proposals.
- Improved candidate relevance without changing governance semantics.

Negative:

- Additional integration surface between ERS and ERE.
- Eventual consistency between governed state and engine hints.

Obligations:

- ERS–ERE contracts must treat projected constraints as optional inputs.
- Integration logic must reject ERE proposals that violate curator-confirmed constraints even if ERE ignores projected inputs.

ADR-F1 — Representation Symmetry at ERS–ERE

ADR-F1 — Representation Symmetry at the ERS–ERE Boundary

Context and Problem Statement

- ERS orchestrates resolution while ERE performs computational matching based on provided representations.
- Divergent or degraded payloads between ERS and ERE introduce hidden coupling, loss of resolution quality, and inconsistent behaviour across engines.
- Payload contracts must remain stable and interpretable even as engines evolve or are replaced.

Decision

- ERE receives the same entity representation and contextual information that ERS uses for resolution.
- There is no degraded, partial, or engine-specific payload model at the ERS–ERE boundary.
- Payload shape is domain-driven and engine-agnostic.
- The ERS–ERE contract specifies data only and does not encode engine heuristics, scoring logic, or algorithmic assumptions.

Scope and Applicability

- This decision applies to all ERS–ERE request and response payloads involved in resolution and re-resolution.
- It governs representation completeness and contract symmetry at the integration boundary.
- Internal engine feature engineering and representation transformation are out of scope.

Considered Options

- Providing a reduced or simplified payload to ERE.
- Defining engine-specific payload variants.
- Using a single, shared, domain-driven representation for ERS and ERE.

Rationale

- Symmetric representations preserve resolution effectiveness and comparability across engines.

- Engine-agnostic payloads support replaceability and long-term evolution.
- Clear data contracts avoid leaking algorithmic assumptions into system architecture.

Consequences

- Payload evolution must be coordinated through domain modelling rather than engine tuning.
- ERS must supply complete and consistent representations to all ERE implementations.
- Engines are responsible for adapting shared representations to their internal processing needs.

ADR-F2 — No Canonical Representation

ADR-F2 — No Canonical Representation Materialisation by ERS

Context and Problem Statement

- Canonical identity and canonical representation evolve at different speeds and serve different purposes.
- Prematurely defining or exposing a canonical entity representation risks coupling identity semantics to specific data shapes or engine outputs.
- Clients and engines may require different representations without redefining what identity means.

Decision

- ERS does not assert, construct, or encode a canonical entity representation as part of the contract.
- Canonical identifiers serve as identity anchors only.
- Any canonical payload produced in the future is treated as opaque output and does not redefine identity semantics or governance rules.

Scope and Applicability

- This decision applies to all ERS–client and ERS–ERE contracts that reference canonical entities.
- It governs what ERS is allowed to expose as authoritative representation.
- Internal representations, engine outputs, and experimental materialisations are out of scope.

Considered Options

- Defining and exposing a canonical entity schema as part of ERS.
- Allowing engines to implicitly define canonical representations.
- Restricting ERS to identity anchoring without canonical payload materialisation.

Rationale

- Decoupling identity from representation preserves long-term flexibility.
- Avoiding canonical payload commitments prevents accidental semantic lock-in.
- Opaque handling of future payloads allows experimentation without contract breakage.

Consequences

- Clients must not assume the existence of a canonical entity representation.
- Identity stability is maintained independently of representation evolution.
- Any future canonical payload must be explicitly introduced without altering identity semantics.

ADR-F3 — UI Preview Extraction

ADR-F3 — UI Preview Extraction is Non-Authoritative

Context and Problem Statement

- Curation interfaces require human-readable previews to support efficient review and decision-making.
- Preview data is derived from underlying representations and may be incomplete, inconsistent, or fail to extract.
- Allowing UI-oriented data to influence resolution or governance risks coupling presentation concerns with core semantics.

Decision

- Preview extraction for curation UI is configuration-driven and best-effort.
- Preview data is presentation-only and has no authority over resolution, governance, or identity semantics.
- Failures or gaps in preview extraction must not block resolution, curation, or corrective workflows.

Scope and Applicability

- This decision applies to all UI-facing preview generation and rendering mechanisms.
- It governs how preview data may be used within curation interfaces.
- Resolution logic, governance state, and persistence are out of scope.

Considered Options

- Treating preview data as authoritative input for resolution or curation.
- Blocking workflows when preview extraction fails.
- Isolating preview extraction as a non-authoritative, best-effort concern.

Rationale

- Separating presentation concerns from core logic preserves architectural clarity.
- Best-effort previews support usability without introducing hard dependencies.
- Non-authoritative previews prevent accidental semantic drift driven by UI needs.

Consequences

- UI behaviour must degrade gracefully when preview data is missing or incomplete.
- Resolution and governance correctness remain independent of preview quality.
- Preview configuration can evolve without affecting core system guarantees.

ADR-F4 — Top-N Pruning Policy

ADR-F4 — Candidate Handling and Top-N Pruning Policy

Context and Problem Statement

- Resolution engines may produce large candidate sets with engine-specific scoring characteristics.
- Applying thresholds at the ERS–ERE boundary risks coupling governance semantics to engine heuristics and losing potentially relevant candidates.
- The system must balance correctness, governance guarantees, and operational scalability.

Decision

- ERE must not filter or discard candidates based on thresholds at the ERS–ERE boundary.
- ERS accepts all candidates returned by ERE as advisory input.
- ERS prunes candidates internally to a configured top-N set for storage, curation, and UI purposes.
- Thresholds are used only internally by ERS for automated acceptance decisions and optionally for UI filtering or bulk actions.

Scope and Applicability

- This decision applies to ERS–ERE contracts, candidate ingestion, and candidate persistence.
- It governs how candidates are filtered, retained, and exposed for curation.
- Engine-internal candidate generation and scoring are out of scope.

Considered Options

- Applying similarity or confidence thresholds at the ERS–ERE boundary.
- Allowing ERE to return only pre-filtered candidate sets.
- Centralising candidate pruning and threshold usage within ERS.

Rationale

- Boundary-level neutrality preserves engine replaceability and governance correctness.
- Centralised pruning enables consistent policy application across engines.

- Separating candidate availability from acceptance decisions maintains the non-empty resolution guarantee.

Consequences

- ERS must handle potentially large candidate sets efficiently.
- Engine-specific scoring differences do not affect candidate availability.
- Threshold tuning can evolve without changing external contracts.

ADR-G1 — No Exposed Version History

ADR-G1 — No Exposed Version History or Lineage Graph for Canonical Entities

Context and Problem Statement

- Clients and integrators often request visibility into historical changes of canonical entities.
- Exposing version history or lineage graphs introduces additional semantics, lifecycle expectations, and coupling to internal state evolution.
- Identity stability and governance guarantees must remain simple and predictable for external consumers.

Decision

- Clients observe only the latest canonical identifier state per request.
- There is no exposed version history of canonical entities.
- There is no canonical lineage graph, merge history, or split history exposed via the contract.
- Internal history or snapshots may exist for audit or operational purposes but are not part of the external contract.

Scope and Applicability

- This decision applies to all client-facing APIs, contracts, and documentation that reference canonical entities.
- It governs what historical information may be exposed externally.
- Internal auditing, logging, and operational diagnostics are out of scope.

Considered Options

- Exposing versioned canonical entities with monotonic version numbers.
- Providing lineage graphs showing merges and splits.
- Restricting clients to the latest canonical state only.

Rationale

- Limiting exposure to the latest state preserves simple and stable client contracts.
- Avoiding lineage semantics prevents scope creep into data provenance and historical modelling.
- Internal flexibility is maintained without committing to long-term external guarantees.

Consequences

- Clients cannot reconstruct historical evolution of canonical entities from the API.
- Audit and traceability requirements must be satisfied through internal tooling.
- Future introduction of lineage or versioning would require explicit new decisions.

ADR-H1 — Observability and Telemetry Semantics

ADR-H1 — Observability and Telemetry Semantics

Context and Problem Statement

- Operating a distributed resolution system requires observability for troubleshooting, performance analysis, and operational insight.
- Telemetry mechanisms must not interfere with core system behaviour or client-facing guarantees.
- Observability requirements evolve over time and vary across deployment environments.

Decision

- Observability follows an OpenTelemetry-style approach for tracing, metrics, and logging.
- Telemetry collection is best-effort and non-blocking and must not affect functional behaviour.
- Telemetry data is metadata-only and does not imply capture of full payloads or sensitive content.
- No compliance-grade auditing or regulatory guarantees are assumed by default.
- Correlation keys align with (sourceId, sourceRequestId, entityType) and internal request identifiers as needed for operational tracing.

Scope and Applicability

- This decision applies to operational telemetry emitted by ERS and related components.
- It governs how observability data is collected, correlated, and interpreted.
- Audit logging, legal compliance, and data retention policies are out of scope.

Considered Options

- Treating telemetry as a mandatory, blocking subsystem.
- Capturing full request and payload data for observability.
- Using lightweight, metadata-only, best-effort telemetry.

Rationale

- Best-effort telemetry avoids coupling system correctness to observability infrastructure.

- Metadata-only collection reduces risk and operational overhead.
- Flexible correlation supports troubleshooting without constraining contracts.

Consequences

- Telemetry gaps may occur and must be tolerated operationally.
- Observability tooling can evolve independently of system contracts.
- Compliance or audit-grade logging requires separate, explicit decisions.

ADR-I1 - Authentication via External SSO

ADR-I1 — Authentication via External SSO; ERS Retains Authorisation Only

Context and Problem Statement

The Link Curation application requires authenticated human access for administrators and curators performing governance actions. Implementing and maintaining a custom authentication mechanism inside ERS or the Link Curation application duplicates existing, well-tested identity infrastructure and introduces unnecessary security and operational risk. A decision is required to clearly separate authentication from authorisation responsibilities while keeping the system architecture simple, secure, and consistent with enterprise environments.

Decision

Authentication for the Link Curation application shall be delegated to an external Single Sign-On (SSO) system.

The following rules apply:

- The SSO system is the sole authority for user authentication and credential management.
- ERS and the Link Curation application shall not manage user credentials, passwords, or authentication flows.
- ERS and the Link Curation application retain authority for authorisation, limited to assigning and enforcing the roles Admin and Curator.
- Access to curation and administrative capabilities shall be granted or revoked by authorisation state managed within ERS.
- Revocation of access shall be effective without undue delay, subject to the guarantees provided by the SSO system.

Scope and Applicability

This decision applies to:

- authentication of human users accessing the Link Curation application,
- validation of authenticated user identity and roles by ERS.

This decision does not apply to:

- machine-to-machine authentication between ERS and external systems,
- identity governance processes within the SSO system,
- user provisioning, approval workflows, or recertification policies.

Considered Options

- Implement local authentication and credential management within ERS.
- Delegate authentication to an external SSO system and manage only authorisation locally.
- Fully outsource both authentication and authorisation to an external identity provider.

Rationale

Delegating authentication to an external SSO system reduces security risk, avoids duplication of established identity functionality, and aligns with common enterprise deployment environments. Retaining authorisation inside ERS preserves clear governance control over who may perform curation and administrative actions, without coupling ERS to external identity governance policies.

Consequences

Positive:

- Reduced security and maintenance burden within ERS and the Link Curation application.
- Alignment with enterprise identity standards and existing infrastructure.
- Clear separation of concerns between authentication and governance authorisation.

Negative:

- Dependence on external SSO availability and behaviour for user authentication.
- Revocation timing is constrained by SSO token and session semantics.

Obligations:

- Architecture and diagrams shall model the SSO system as an external supporting system.
- User management use cases shall be limited to authorisation and role assignment, not credential handling.
- ERS–Link Curation integration shall validate authenticated identity and roles based on SSO-issued assertions.